flint front ends

Four ways to put an S3 bucket in front of a workload — **flint-lite**, **flint-lean**, **flint-passthrough** and **flint-forge** — with the multi-cluster and security consequences of each, the one durable store they all share, and how they compose. In one word each: lite is **NFS**, lean is **sync**, passthrough is **FUSE**, forge is **git**.

Scope. The architecture of the four front ends as built, the fleet shape each implies when one user launches agents across several Kubernetes clusters, what identity is actually enforced on the data path, the security posture of each, where S3 sits in all four, and the rule under which they can be used together. Every claim here is drawn from the repository at the revision noted below; where a guarantee has a stated ceiling, the ceiling is stated rather than rounded up, and where a drill refuted a claim the drill wins.

Four technologies. The word names what carries the bytes between the workload and the bucket, and it is the first row of the table on page 13. **flint-lite is NFS**: a hub pod serves the tree over NFSv4.2 and the node's kernel client carries every byte. **flint-lean is sync**: the agent writes an ordinary local directory, and a sidecar process copies changed bytes to S3 with the plain S3 API, after the fact — the way rsync or Dropbox are described. Nothing intercepts the agent's file operations, which is the defining property of FUSE, and nothing mounts the bucket, so "indirect FUSE" would be wrong; flint-lean has no FUSE anywhere. The chunked manifest, the content-addressed chunks and the barrier are how lean makes that reconciliation atomic and cheap, but they are properties of the sync, not a different mechanism. The two-word names that also fit are *local-first sync* and *S3 publisher*. **flint-passthrough is FUSE**: Mountpoint for S3 intercepts every file operation in the pod and translates it into object requests, so the bucket is what the pod touches. **flint-forge is git**: real git, served per repository, with S3 as its only durable state.

Sources of record. `docs/plans/csi-node-mount-design.md` (the CSI delivery), `docs/plans/flint-lean-plan.md`, `docs/flint-lite-architecture.html`, `docs/flint-lean-architecture.html`, `docs/plans/flint-forge-design.md` (§13 for the falsifiers as run, §17 for composition), `docs/plans/flint-forge-simplification-2026-09-05.md`, the four charts, the drills under `forge/e2e/` (`scale/README.md` for the large-repository campaign, `latency/`, `repack/`), and the code under `spdk-csi-driver/src/{s3csi,passthrough,tier,lite_operator,lean_operator,forge_operator,lite_gateway}`, `lean/sidecar/src` and `forge/syncer/src`. The security page also draws on the approach radar's verified Security cells (`docs/radar/`).

Regenerating. `docs/architecture/diagrams.py` writes every diagram; `docs/architecture/build.sh` validates, rasterises and renders. The diagrams are standalone SVG files under `diagrams/`, referenced rather than inlined, so they can be reused in any other format; the build also emits a Markdown rendition for conversion. Colour means one thing in them: which front end — the same four hues the approach radar uses.

Four front ends, one bucket

flint puts a bucket in front of a workload four different ways. They are not four configurations of one thing and not four tiers of the same product: they are four different bargains, each coherent with itself, over one bucket. Three of them are file-shaped and share a format; the fourth is git-shaped, and stores a bare repository. What varies is how much the workload gets and how much machinery has to exist for it to get that. In one word each, the technology that carries the bytes: lite is **NFS**, lean is **sync** (local disk with an S3 sync sidecar — no FUSE anywhere), passthrough is **FUSE**, forge is **git**.

flint-lite — the hub

NFS · live shared POSIX · many readers and writers

the consumer pod — any cluster

nothing from flint is installed in it
`/data  (an ordinary PVC)`
the node's kernel NFS client carries every byte — one mount per node, shared with its page cache
full POSIX: byte-range locks, atomic rename, O_EXCL

the hub — ONE pod per volume

`flint-pnfs-mds · mode: standalone`
`:2049  NFSv4.2 — the data plane`
`:8080  status + file API — ClusterIP only`
the PVC is the working set and a CACHE of the bucket, never the only copy
single replica · Recreate · exclusive flock

coordination — the hub IS the authority

leases, locks and close-to-open are enforced in one process, so every client sees one coherent tree; a second hub on the prefix is fenced by the epoch cell

publish · hydrate
RPO = the flush cadence

flint-lean — checkout / publish

sync · one agent, one workspace, one writer

the agent pod

the app image is unchanged and holds no S3 credential
`/workspace  (local disk — an emptyDir)`
plain files with no interception anywhere in the read or write path
git, sqlite and build caches work, at disk speed

flint-sync — an unprivileged worker

`delivered by the s3.csi.chert.us driver`
checkout at start, before the agent's first line; a barrier at cadence from an mtime/size scan
`.flint/publish  →  .flint/publish.ack`
preStop drains — a graceful stop loses nothing

coordination — a lease in the BUCKET

one writer per subtree, admitted by a CAS on the epoch cell; a second syncer is REFUSED, never merged; readers see the last published boundary

checkout · publish
RPO = the last barrier

flint-passthrough — the mount

FUSE · an S3 prefix mounted as-is · no flint semantics

the tenant pod

no sidecar, no label, no credential, no privilege — admissible under PodSecurity restricted
`/mnt/s3  (a csi: volume naming a CR)`
objects appear as files, served by Mountpoint for S3
NOT POSIX: no rename, no append, no in-place write

s3.csi.chert.us — a CSI node DaemonSet

one privileged process per NODE, not per pod: it opens /dev/fuse and calls mount(2) itself, then hands the fd to an unprivileged worker running an unchanged mount-s3
the plugin holds no S3 credential — a broker mints them

coordination — NONE, by design

no cache, no manifest, no publish boundary, no coordination between pods sharing a prefix; S3's own consistency is the entire contract

GET · PUT
no RPO — nothing is buffered

flint-forge — the git server

git · a remote per repository · S3 behind it

the agent pod

a stock git client; the working tree is local disk
`credential.helper = flint-forge-credential`
the pod's projected ServiceAccount token, audience forge.chert.us, is the Basic password
the durable unit is a commit — a push

the door, and one server pod per repository

the door is lite's gateway with a git arm: TokenReview, consumers, routing, wake
`gitcgi + git-http-backend + one syncer`
the local repository is an emptyDir CACHE; the pod idles to zero
one syncer owns every write to the bucket

coordination — one CAS per batch

a push is acknowledged only after its pack and ONE CAS'd snapshot are in S3; a stale push is refused by name; a merge is a push to refs/for/<target>

push · restore
RPO = the last acknowledged push

One bucket. Four layouts — and a volume can move between the file-shaped three.

Whole-file objects plus a per-front-end control namespace; for forge, a bare git repository that git can clone read-only with the server down.

flint-lite writes

`<prefix>/<path>`
`<prefix>/.flint/epoch · manifest · owner`

flint-lean writes

`<prefix>/files/<path>`
`<prefix>/.flint/lean/epoch · claim`
`<prefix>/.flint/lean/current · inbox`

flint-passthrough writes

`<prefix>/<key>`
and nothing else — no control namespace, because there is no state to keep

flint-forge writes

`<prefix>/git/objects/pack/*`
`<prefix>/git/snapshot (CAS) · epoch · claim`
`<prefix>/lfs/objects/<oid>`
`<prefix>/files/<path>  — the export`

Read the columns as an escalation ladder, cheapest first: passthrough asks nothing of the workload and gives it nothing back; lean gives real POSIX at local speed to one writer; the hub is the only one where several pods share one live tree; forge adds history, a merge policy and an acknowledgement that means durable — for code.

flint-lite · flint-lean · flint-passthrough · flint-forge · data path · control · hazard / trust boundary

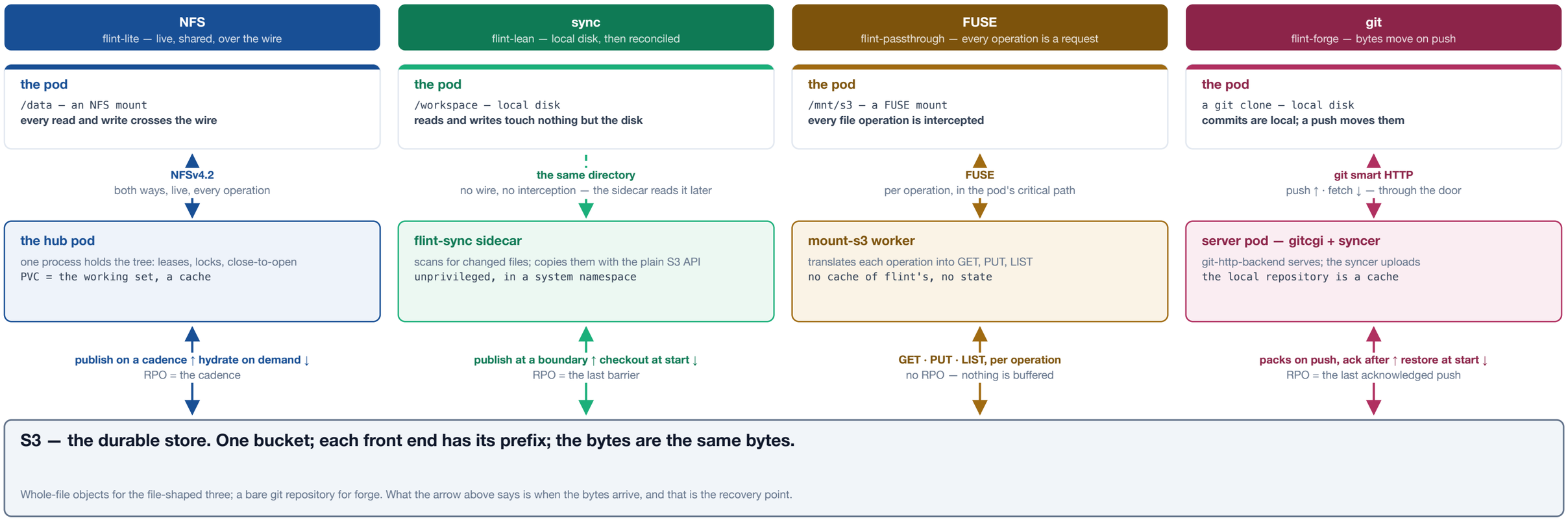
A box wears its front end as the strip on its top edge. S3 and Kubernetes are neutral: they belong to nobody.

Read the columns as an escalation ladder, cheapest first. Passthrough asks nothing of the workload — nine lines of pod spec — and gives it nothing back beyond the objects: no POSIX, no coordination, no boundary. **Lean** gives a real local filesystem at disk speed to one writer, and a durable point that writer can name. **The hub** is the only one where several pods, in several clusters, can share one live tree with locks that mean something. **Forge** stands slightly apart: it is for code, its unit of durability is a commit, and what it adds is history, a merge policy the server enforces, and an acknowledgement that means the bytes are in S3.

The common floor is the bucket format, and it is what makes the file-shaped choice reversible. Lite, lean and passthrough all write whole-file objects; what differs is the `.flint/` control namespace beside them, which exists to answer questions the object listing cannot — who holds the write lease, what the last coherent boundary contained, what the whole tree is for a restore. Passthrough writes no control namespace at all, which is exactly why it can be pointed at a prefix somebody else's tooling already owns. Forge's bucket is a bare git repository plus one CAS'd pointer, and it joins the family through its export: a ref's tree published, by the shipped lean binary, as a valid lean workspace. Because the file formats are common and mutually convertible, a volume that outgrows one front end can move to another rather than being re-plumbed — and page 12 shows that the better question is usually not which one, but which one for which data.

How the bytes move

The same four front ends, drawn as flows and nothing else. Read each column top to bottom: what the pod touches, what carries the bytes, and what puts them in the bucket. Solid arrows carry file contents; the dashed boxes are control — leases, tokens, keys — and never carry a file.



Solid arrows carry file contents; the dashed arrow is a shared directory, not a wire. The word on each header is the technology between the pod and the bucket.

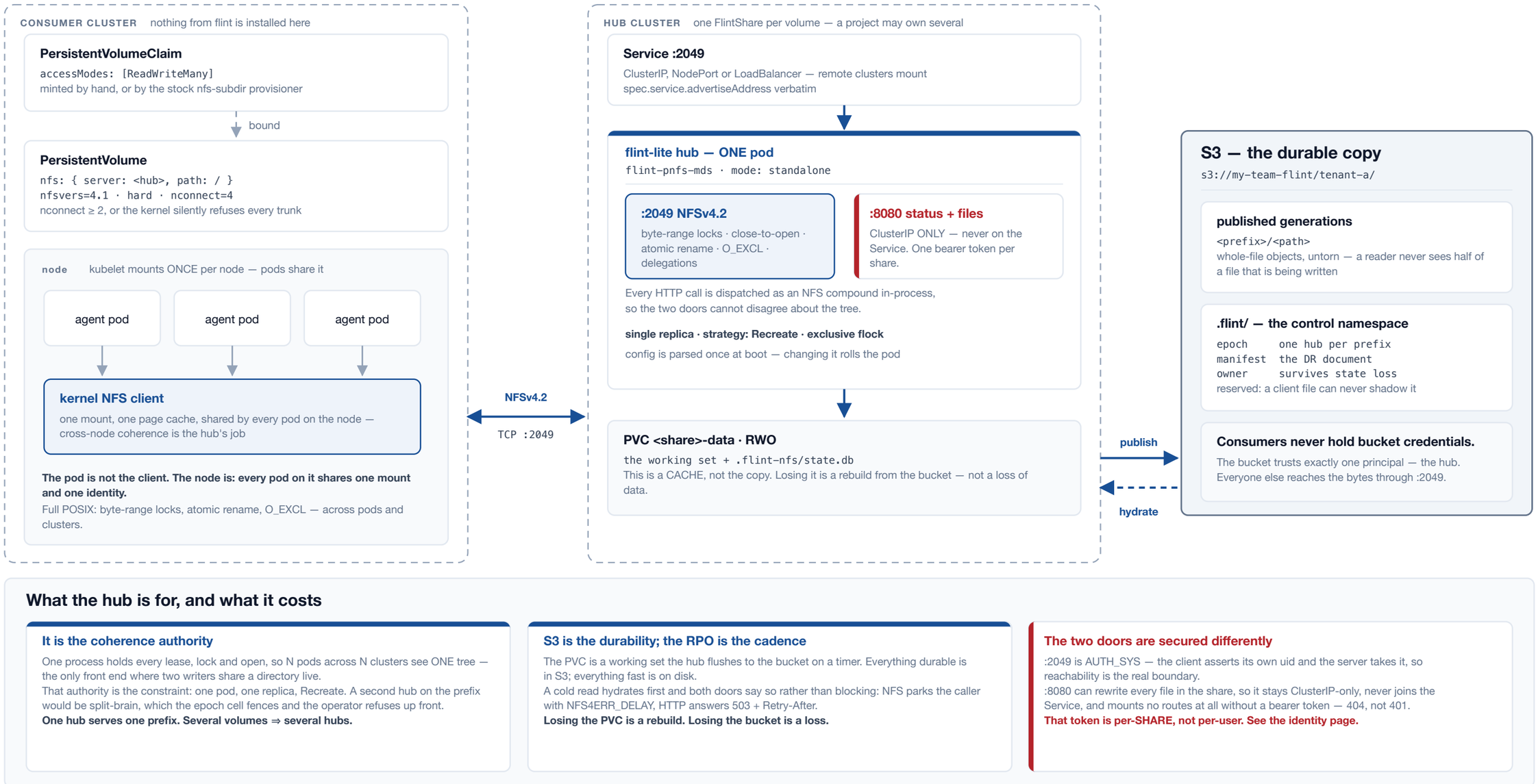
flint-lite · flint-lean · flint-passthrough · flint-forge · data path · control

Two of the four put nothing between the pod and its files. A lean pod writes ordinary local disk, and the sidecar reads that same directory after the fact — the pod is never blocked on S3, never sees a credential, and never waits. A forge pod holds an ordinary clone; bytes leave it only when it pushes. The other two intercept: the hub's NFS mount carries every read and write over the wire to one process that holds the tree, and passthrough's FUSE mount turns every file operation into an object request in the pod's own critical path.

When bytes reach the bucket is the recovery point, and the column says it. Lite publishes on a cadence and hydrates a cold read on demand; lean publishes changed files at a boundary and checks out at start; passthrough has no moment of its own, because every operation is the request; forge uploads packs on push and reports `ok` only after the snapshot CAS lands, and restores at start. Everything in the dashed boxes decides who may do these things, and none of it is on the path the bytes take.

flint-lite — the hub

One pod serves real POSIX over NFSv4.2 to any number of pods in any number of clusters. The PVC beside it is a working set; an S3 prefix is the durable copy. A second listener exposes the same filesystem over HTTP for callers that cannot mount. One hub serves exactly one prefix — a project that needs several volumes runs several hubs.



What the hub is for, and what it costs

It is the coherence authority

One process holds every lease, lock and open, so N pods across N clusters see ONE tree — the only front end where two writers share a directory live.
That authority is the constraint: one pod, one replica, Recreate. A second hub on the prefix would be split-brain, which the epoch cell fences and the operator refuses up front.
One hub serves one prefix. Several volumes ⇒ several hubs.

S3 is the durability; the RPO is the cadence

The PVC is a working set the hub flushes to the bucket on a timer. Everything durable is in S3; everything fast is on disk.
A cold read hydrates first and both doors say so rather than blocking: NFS parks the caller with NFS4ERR_DELAY, HTTP answers 503 + Retry-After.
Losing the PVC is a rebuild. Losing the bucket is a loss.

The two doors are secured differently

:2049 is AUTH_SYS — the client asserts its own uid and the server takes it, so reachability is the real boundary.
:8080 can rewrite every file in the share, so it stays ClusterIP-only, never joins the Service, and mounts no routes at all without a bearer token — 404, not 401.
That token is per-SHARE, not per-user. See the identity page.

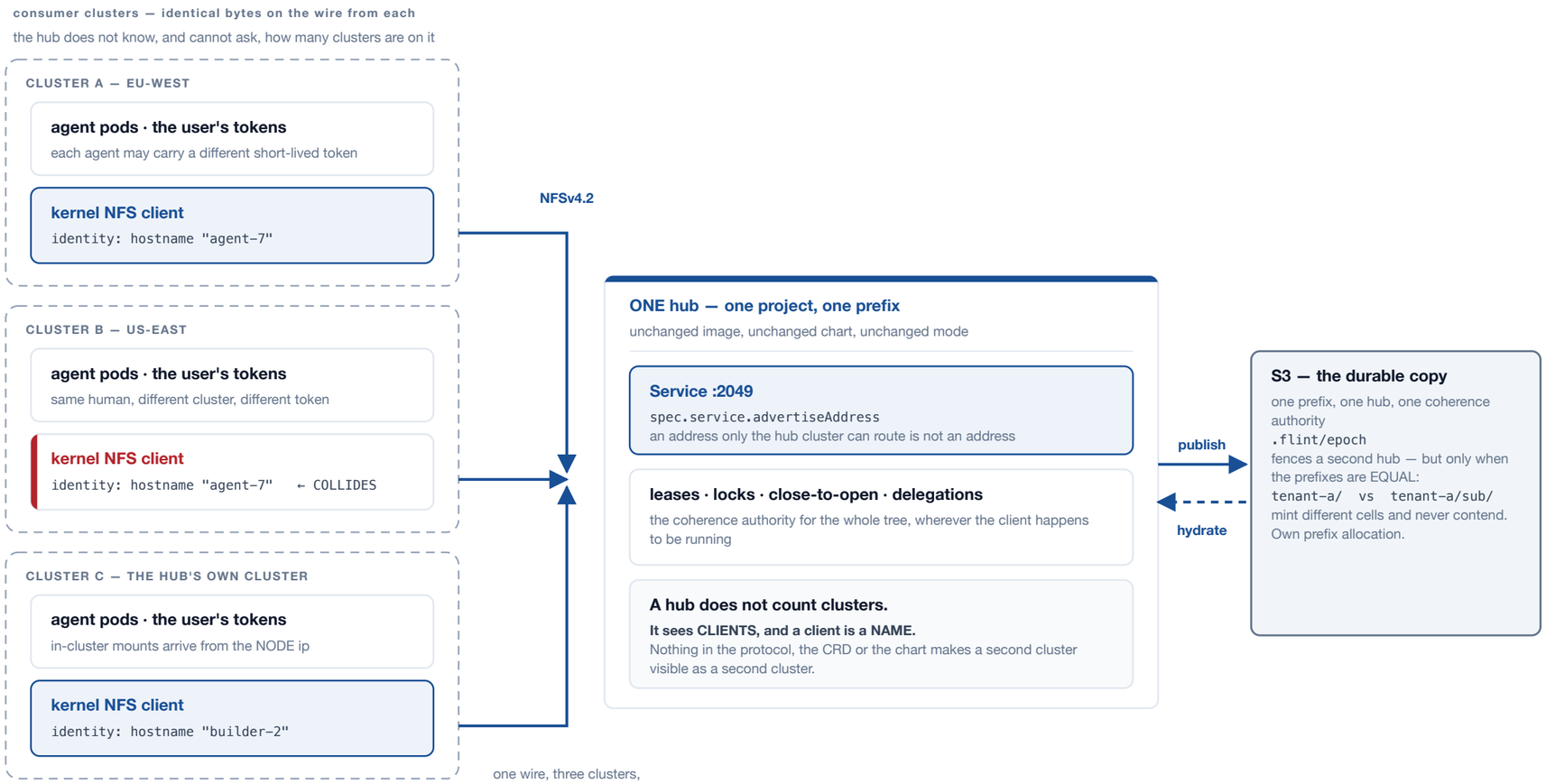
data path · · · control ▮ hazard / trust boundary

The hub's value and its constraint are the same fact. One process holds every lease, lock and open, so N pods across N clusters see one coherent tree — and that is why it is one pod, one replica, `Recreate`, behind an exclusive flock. Two hubs on one prefix would be split-brain, which the store-side epoch cell fences and the operator refuses up front. Consumers install nothing: the data path is the node's own kernel NFS client, one mount per node, shared by every pod on it along with its page cache.

The disk is a cache; the bucket is the copy. The hub flushes the working set to S3 on a cadence, so the recovery point is that cadence and losing the PVC is a rebuild rather than a loss. A cold read hydrates on demand and both doors say so rather than hanging — NFS parks the caller with `NFS4ERR_DELAY`, HTTP answers `503 + Retry-After`. **The two doors are secured differently, and only one of them has a credential.** Port 2049 is AUTH_SYS, so the client asserts its own uid and reachability is the real boundary; port 8080 can rewrite every file in the share, so it stays ClusterIP-only, never joins the Service, and mounts no routes at all without a bearer token — `404`, not `401`. That token is per-share, not per-user, which page 8 takes up.

flint-lite across clusters

One hub can serve agents in several clusters at once. The wire is ordinary NFSv4.2 and the hub does not know, or care, how many clusters are on it — which is precisely the problem. Three things the protocol and the chart cannot supply become the fleet's responsibility, and every one of them fails silently if it is missed.



What the FLEET must supply

three things neither the protocol nor the chart can do for you

1 A unique name per client

The identity is the hostname and nothing else. Two clusters sharing a node name are ONE client, and RFC 8881 says the server MUST read the second as the first REBOOTING — so the incumbent's locks and opens are dropped, silently, correctly, on a false premise.

```
hostname = <cluster>-<node>
nfs.nfs4_unique_id=<cluster>
```

2 A boundary at the network layer

Port 2049 is AUTH_SYS, so reachability IS the boundary — and networkPolicy cannot draw it: kube-proxy SNATs a NodePort or LB client to an address in the HUB's cluster before the packet arrives.

Measured: 1486 of 1486 connections from two remote clusters arrived as the hub cluster's gateway — none as a remote node.

Use peering, security groups or a gateway.

3 A keepalive per remote mount

The idle ladder suspends a quiet share, and a partitioned cluster's busy agents look exactly like quiet ones. Drive POST /wake from the thing that knows whether work is happening.

suspendWithSessions DEFAULTS to suspending anyway.

The image is unchanged. The operating contract is not.

Admission uniqueness stops at the cluster edge

The operator refuses a second share on one prefix from its own cache — CEL and policy see one object at a time. Two clusters each admit a share on one prefix and neither sees the other; the store-side epoch is the only backstop, and it catches EXACTLY equal prefixes.

Addresses are not arbitrated either

Two shares may advertise the same address and both report `Conflict=False (Unique)` — that condition covers bucket-prefix overlap. A true statement about STORAGE that says nothing about addresses.

The failure mode is silent by construction

A colliding client gets NO error. The protocol says it is the same client returning, so the server does exactly the right thing with a false premise, and the loser's state is simply gone.

Six defects in v1.37.0 traced to this one cause.

data path — — — control ▮ hazard / trust boundary

A hub does not count clusters — it sees clients, and a client is a name. The NFSv4 client identity is the hostname and nothing else, so two clusters that share a node name are one client to the hub, and RFC 8881 §18.35.5 requires the server to read the second as the first rebooting: the incumbent's locks and opens go, silently, correctly, on a false premise. Six defects in v1.37.0 traced to this one cause. Fix it with `hostname = <cluster>-<node>`, or the client-side kernel parameter `nfs.nfs4_unique_id`, which survives a rename.

The reachability boundary cannot be drawn where you would expect to draw it. kube-proxy SNATs a NodePort or LoadBalancer client to an address in the hub's own cluster before the packet arrives: measured on a three-cluster rig, 1486 of 1486 connections from two remote clusters arrived as the hub cluster's gateway, none as either remote node. Listing remote CIDRs in `networkPolicy` is an outage; listing the local one admits everyone. Use peering, security groups, or a gateway. **And a partitioned cluster's busy agents look exactly like idle ones** — `suspendWithSessions` defaults to suspending anyway, and even when set it only narrows the window, because leases expire. Drive `POST /wake` from the thing that actually knows whether work is happening.

flint-lean — checkout and publish

If the workspace fits on disk, no interception is needed at all. Materialise plain files before the agent's first line runs, publish changed files at cadence or at a boundary the agent declares, and keep the write authority out of the pod. Most of a FUSE front end's value at a fraction of its risk surface — same bucket, same engine, no kernel path.

The loop — one agent, start to durable

1. **pod starts**
   checkout runs first — the agent never sees an empty tree
2. **the agent works**
   local files at local speed; the syncer publishes at cadence anyway
3. **the agent declares a boundary**
   `echo > .flint/publish`
   one file write — the entire API surface
4. **the worker answers**
   `.flint/publish.ack`
   ok ⇒ the bytes are in S3 · partial · refused-fenced

Gated mode — durability and visibility become separate events

the upload lane — always running

every changed file becomes a NEW OBJECT VERSION as soon as it is scanned. Durable immediately —
and INVISIBLE to readers.
A crash in this lane loses nothing, because nothing here was ever cited.

the citation — ONE CAS

installs the whole pending set at once, naming version ids. No copy phase, so no half-boundary exists.
A reader sees the whole change or none of it.
Gated is REFUSED without a lag bound — unbounded staleness is impossible by construction.

Three questions, in order — the cheapest disqualifier first

1 · Does the workspace fit the disk?

The whole tree is materialised — no stubs, no eviction — and the gate is the FILE COUNT before the byte count: ~250k files, which falls out of the manifest size.
No ⇒ this is not the front end you want.

2 · Is one writer per subtree enough?

The lease admits exactly one syncer; a second is REFUSED, not merged. Two agents cannot share one live tree here — several integrate through a git host instead.
No ⇒ the hub, for live shared POSIX.

3 · Is snapshot freshness acceptable?

Readers see the last published boundary, never your last write. A hard kill loses at most floorSecs (default 60 s); a graceful stop loses nothing.
No ⇒ the hub. Yes ⇒ lean is the cheapest thing that works.

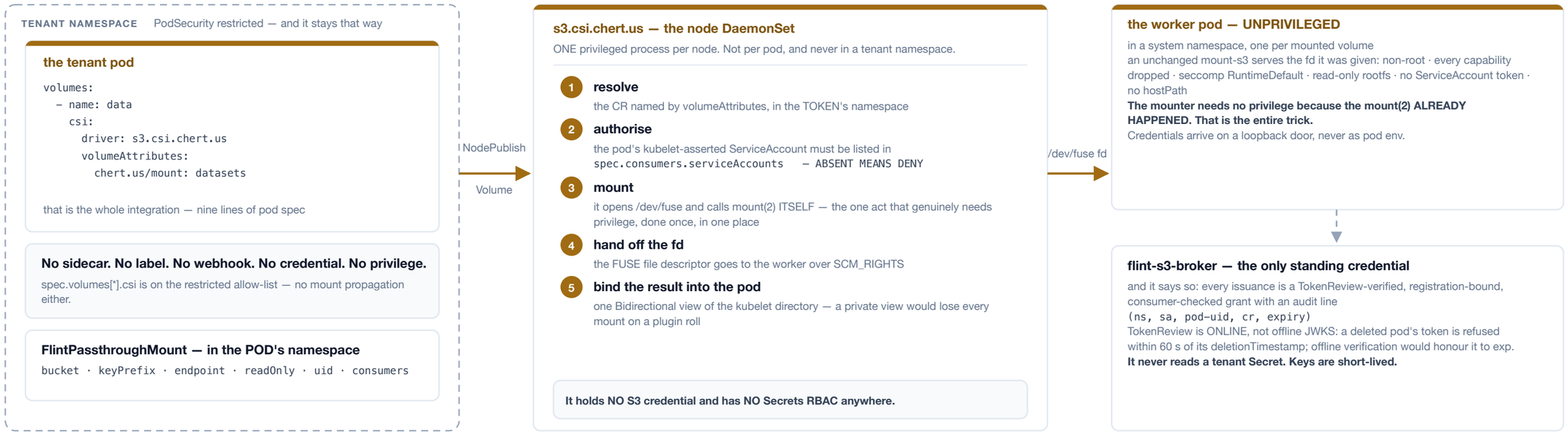
data path · · · control hazard / trust boundary

The ordering is the architecture. kubelet blocks every container in the pod on `NodePublishVolume`, so the checkout completes before the agent's first line runs — no init container, no readiness dance — and calls `NodeUnpublishVolume` only after every container has exited, which is where the final barrier goes. In between, the app touches plain files on real local disk with zero interception, so git, sqlite, hard links and index locks simply work. The app container holds no S3 credential; the unprivileged worker does, in a system namespace, behind a loopback door the app cannot reach.

The boundary verbs are files, because the workspace is the interface. An agent that can write a file can declare a coherent point — `echo > .flint/publish` — and learn when its bytes are durable, from `.flint/publish.ack`: `ok` means the bytes are in S3, `refused-fenced` means this syncer lost the lease and is saying so rather than leaving the agent waiting forever. No client library, no credential, no network path. **Gated mode answers a different question from cadence:** cadence asks how stale a reader may be, gated asks whether a reader may see half a logical change — and answers no, by uploading every changed file as a new version immediately (durable, and invisible) and installing the whole pending set with one CAS. Gated is refused without a lag bound, so unbounded staleness is impossible by construction rather than by convention.

flint-passthrough — the CSI mount

An S3 prefix, mounted into a pod as-is by Mountpoint for S3, with no flint semantics layered over it. There is no controller and no status, because nothing about a passthrough mount converges. What there is, is a careful answer to the question of who holds privilege — and the answer is: not the tenant.



S3 — and it is just S3

This is the one front end that writes NO control namespace at all.
`<prefix>/<key>`
Objects appear as files and nothing else is written — no manifest, no epoch cell, no claim, no publish boundary. A prefix somebody else's tooling already owns stays exactly as it was.
Durability, consistency and access control are S3's, unmediated. flint adds none and removes none.

What a passthrough mount is NOT

every one of these is a design decision, not a gap waiting to be filled

It is not POSIX.	No rename, no append, no in-place modification — at any setting.
It is not coordinated.	Two pods on one prefix do not see each other. There is nothing to see with.
It is not per-user.	The mount presents as ONE uid — NodePublish never sees the pod's securityContext.
It is not self-healing.	If the mounter dies, running containers are stranded on ENOTCONN; recreate the pod.

Where the privilege went — the accounting, said plainly

The webhook this replaced put a privileged container into every tenant pod. Privilege did not disappear; it was concentrated, the way Istio's CNI move concentrated it.

flint-s3-csi-node — privileged
One pod per node. It needs /dev/fuse and a Bidirectional mount of /var/lib/kubelet, which the API server permits on privileged containers only. It may create and delete pods in ONE namespace, and — with the admission policy — only on its own node.

worker pods — unprivileged
One per mounted volume, in the workers namespace. The namespace carries the PodSecurity privileged label ONLY because hostPath is baseline-forbidden — the ValidatingAdmissionPolicy is the real guard.
Read the label as a formality, and the policy as the fence.

the tenant pod — nothing
It is given no privilege, no credential and no flint code. Its ServiceAccount identity is asserted by kubelet, which the pod cannot forge, and checked by the broker online.
The same chain flint-lean uses: the two differ in what runs in the worker, not in who is trusted.

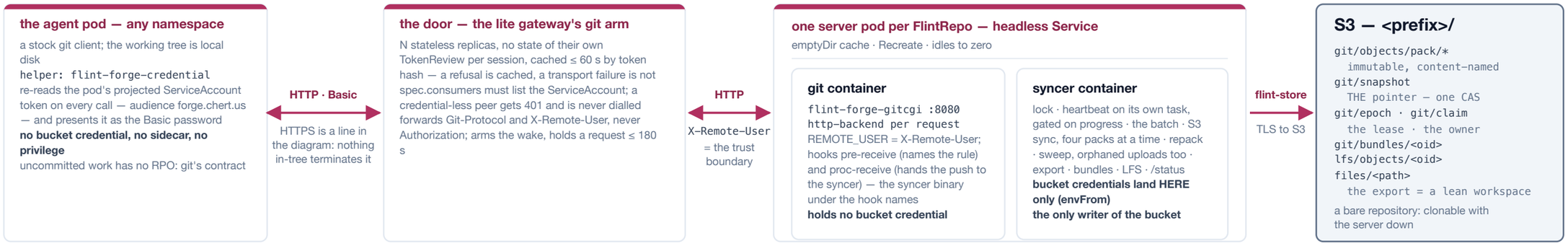
data path · · · control ▮ hazard / trust boundary

The trick is that the mount happens before the mounter runs. The node plugin opens `/dev/fuse` and calls `mount(2)` itself — the one act that genuinely needs privilege, done once, per node — then hands the file descriptor to an unprivileged worker pod over `SCM_RIGHTS`, where an unchanged `mount-s3` serves it needing no privilege at all. The tenant pod is given nothing: no sidecar, no label, no webhook, no credential, no privileged container. It declares a `csi:` volume naming a CR in its own namespace and stays admissible under PodSecurity `restricted`.

Privilege did not disappear; it was concentrated. It lands in exactly three places, none of them a tenant namespace: the node plugin (privileged, one per node, holding no S3 credential and no Secrets RBAC), the worker pods (non-root, all capabilities dropped, read-only rootfs, no ServiceAccount token), and the broker (the only standing credential, and every issuance is TokenReview-verified and audit-logged). **What a passthrough mount is not** is worth stating as plainly as what it is: not POSIX — no rename, no append, no in-place modification at any setting; not coordinated — two pods on one prefix do not see each other; not per-user — the mount presents one uid, because `NodePublish` never sees the pod's `securityContext`; and not self-healing — if the mounter dies, running containers are stranded on `ENOTCONN` and the pod must be recreated. A workload that needs any of those wants flint-lean.

flint-forge — the git server

A git server per repository, with S3 as its only durable state. Agent pods are stock git clients; a stateless door turns the pod's own ServiceAccount token into a principal and routes to the repository's server pod, where real git — `git-http-backend` behind a small CGI runner of flint's, the standard hooks — serves from a local disk that is a cache, and one syncer owns every write to the bucket. The design's rule was to reinvent nothing git already does; what flint supplies is exactly what git lacks in a cluster with an object store behind it.



The push — acknowledged means durable

receive-pack serialises nothing and, under proc-receive, checks nothing: the syncer re-implements staleness and fast-forward, and batches every push that arrives while one is in flight.

1 the hook hands the commands over
one process per push; the syncer judges them in arrival order against the local refs, the snapshot AND its own running view; a stale one is ng stale: fetch first

2 policy, twice
pre-receive names the rule at the edge; the syncer enforces the same document, so a missing hook does not open the repository

3 renew the lease
a foreign holder ⇒ 412 ⇒ this server fences itself: no reads, no writes, exit

4 upload the packs
unconditional PUT of content-named, immutable objects — a merge's loose objects are packed first

5 ONE snapshot CAS
If-Match on the etag: thirty refs move as one object, and a reader never sees half a batch

6 the ref transaction, then the report
update-ref --stdin applies locally; only THEN ok or ng per ref. A crash before this fails every push at the client and the restart restores from the snapshot

Idle to zero — one rung, and a wake the door holds

idle for suspendAfterSecs ⇒ replicas 0 and the emptyDir is gone, so a parked repository costs nothing but its objects. The ladder suspends only on the server's own polled /status, and a poll failure holds forever.
The door arms chert.us/requested-at on the CR, waits on the CR — never the pod — and holds the request up to 180 s; the restore is one pack.
As built the clock counts pushes only: a fetch never reaches the syncer, so read-only use is parked one threshold after its wake.

Fleet levers — take the server out of the transfer

A thousand clones are ~130 CPU-s but 43 GB from one NIC: egress binds before CPU. Bitmaps ship with every pack; a clone BUNDLE is cut on a floor, uploaded beside the packs and advertised as a presigned URL re-signed at half its TTL — the client must opt in (transfer.bundleURI=true).
Measured on EC2 (F8): 1,000 clones cost the server 5.7 MiB of egress with bundles advertised and the client opted in, 40,409 MiB with the opt-in off — 7,000×.
LFS: the batch API runs in the syncer and answers with presigned URLs, so a checkpoint goes client-to-store and the pod sees a few hundred bytes of JSON. The pruner takes only branches main already contains, and only once quiet.

The export — a legible mirror, by the shipped lean binary

After a batch moves an exported ref, the syncer materialises the tree with a two-tree read-tree — touching exactly what changed, removing what is gone — and runs flint-sync barrier over it, so lean's ordering (upload, CAS, deletes LAST) is inherited, not re-derived.
It runs AFTER the report and never CASes the snapshot; lean and passthrough readers mount main read-only with no forge code in them.
A mirror that is never repaired: a foreign write into it is not noticed (the barrier diffs a local scan against a local baseline); a reader that verifies against the manifest refuses it, a mount with no manifest takes it. Readers of an export are readers.
It runs inline in the serving loop: a blocked barrier times out and backs off, and the heartbeat beats through it on its own task — but pushes stall for that long.

Built, and drilled on real clusters — 2026-09-04 and 2026-09-05

Twelve falsifiers green: eleven on EC2 on 2026-09-04 — durable under 8 mid-push kills, two pushes to one ref, a merge across a cold restore, the fence, a byte-identical restore and a dumb clone with the server down, protected main, idle-to-zero, the storm, the export, the sweep, an S3 outage — and per-principal rights on Cilium on 2026-09-05, 17 legs. Gitea's 4,019 MiB of egress for 100 clones sits level with forge's control arms.
The large-repository campaign, 2026-09-05, on real S3: a 10 GiB push acknowledged in 262 s with the lease token silent past the 60 s window in both the push and the restore, so a challenger took a live repository mid-restore — durability held, availability paid. Fixed the same day: the heartbeat on its own task, gated on progress; the restore a bounded fan-out over ranged GETs (297 MiB/s at 40 GiB, RAM flat); orphaned uploads swept; every claim rotates the snapshot (two gaps found by the TLA+ model); a pack listed only once its index lands; one runner in place of nginx and fcgiwrap.
Confirmed: 40 GiB in 1113 s; a challenger beside a live restore never claimed; 40/40 pushes durable; five stalled pushes and eight clones at once.
Open: a pack's parts upload one at a time; a full repack every 24 pushes re-uploads the repository; a roll mid-push costs that push. Not drilled: agents in another cluster. Not forge: a web UI, pull requests, code search, CI, a POSIX volume, untracked files.

data path control hazard / trust boundary

Acknowledged means durable, and the mechanism is one syncer per repository. The review's central finding was that `receive-pack` serialises nothing: under `proc-receive` git performs neither the old-oid check nor the fast-forward test for the commands it hands off, so two concurrent pushes to one ref run their hooks fully overlapped. The syncer therefore re-implements both checks — against the local ref, the bucket's snapshot *and* its own running view of the batch — renews the lease, uploads the packs, CASes the one mutable object (`git/snapshot`, which names every ref, pack and bundle) and applies the ref transaction, and only then reports `ok` or `ng` per ref. A stale push is refused by name; a merge is a push to `refs/for/<target>` that the server performs with `merge-tree`; a crash before the report fails every push in the batch at the client, and the restart restores from the snapshot. Policy is enforced twice from one document — `pre-receive` names the rule, the syncer enforces it — so a missing hook does not open the repository.

The levers are about taking the server out of the transfer. A thousand clones cost about 130 CPU-seconds but 43 GB from one NIC, so egress binds long before CPU does; measured on EC2 at 1,000 clones, the server's egress was 5.7 MiB with clone bundles advertised and the client opted in (`transfer.bundleURI=true`), against 40,409 MiB with the opt-in off — the storm falsifier's own arms; phase 0's 100-clone comparison put Gitea, the buy option, at 4,019 MiB, level with forge's controls. LFS goes the same way — the batch API runs in the syncer, which has the bucket credentials the door deliberately does not, and answers with presigned URLs, so a checkpoint never crosses the pod. Idle is one rung: no pushes for `suspendAfterSecs` and the replica count goes to zero; the door arms a wake annotation on the CR and holds the request, which on the cluster meant a clone through a parked repository completed in 11 s on a pod the request itself created. The export publishes a ref's tree as a real lean workspace by running the shipped `flint-sync` over it, after the report and never as a second writer of the snapshot.

The edges, stated as the drills left them. Eleven falsifiers ran on EC2 on 2026-09-04 and went green, two of them after finding a defect (a fresh repository could not create `main`; the export froze on a parked upload). A twelfth, per-principal rights and the header boundary, ran on kind with Cilium on 2026-09-05, 17 legs green: the `X-Remote-User` boundary holds only while the operator's NetworkPolicy stands. Nothing in-tree terminates TLS, so the pod's audience-bound token rides two cleartext hops inside the cluster; the built idle clock counts pushes only, so read-only use is parked one threshold after its wake; and the server runs as root in both images. An export is a mirror that is never repaired — page 12.

The large-repository campaign of 2026-09-05 moved the ceiling, and the fixes landed the same day. On real S3 a 10 GiB push was acknowledged in 262 s with the lease token silent past the 60 s takeover window, so a challenger took a live repository mid-restore; durability held, availability paid. Since then the heartbeat is its own task, gated on progress; the restore is a bounded fan-out over ranged GETs, 297 MiB/s at 40 GiB; orphaned uploads are swept; every claim rotates the snapshot, closing two gaps a TLA+ model found; and a pack is listed only once its index has landed. Confirmed on the wire at 40 GiB with a challenger present; what stays open is on the plate.

Where a user's token can and cannot go

One human launches agents across clusters A, B and C. Each agent holds that user's token — short-lived, from an IdP, and possibly a different token in each cluster. This page answers the only question that matters for isolating one user's work from another's: what does storage actually check, and where could that token be presented if you wanted it to be?

The premise — one human, many clusters, many tokens

One end user launches agents on clusters A, B and C; each agent holds THAT user's token, short-lived, from an IdP, and they may all differ. Where does it go, and what does storage actually check?

The answer, for all four: it does not reach the data path. Every front end authenticates the WORKLOAD, never the human behind it.

flint-lite

ONE hub, reached from every cluster

Nothing on the wire carries a user identity.
NFS has no field for a bearer token and no place to add one. krb5 exists in the server, is not surfaced by the chart, and is not an IdP token anyway.

what the agent holds

the user's token — and it never sends it anywhere, because no door on the data path would read it
The pod is not even the client. The NODE is: one mount and one identity per node, asserting its own uid and hostname

the door — :2049, AUTH_SYS

The client asserts its own uid and the server takes it. No token is validated because none is presented.
`security.enforcePermissions: false` is the default — the mode is evaluated, LOGGED, and the operation allowed anyway; even enforced, the hub holds CAP_DAC_OVERRIDE.
sec=sys buys IDENTITY, not ENFORCEMENT.

what the hub sees

a client NAME — the hostname — and a self-declared uid. Not a user, not a cluster: two clusters sharing a node name are ONE client, and the protocol drops the incumbent's locks silently.
The file API's bearer is per-SHARE, not per-user.

what isolates user A from user B

Nothing inside the hub.
Draw the boundary outside it: one hub per project — one CR, one prefix, one coherence domain — and network reachability by peering, SGs or a gateway. networkPolicy cannot draw it.

flint-lean

one syncer per workspace, in every cluster

The identity IS checked — it is just not the user's.
One CSI driver and one broker PER CLUSTER, because TokenReview runs against the LOCAL API server. Clusters meet in the bucket, not at each other's brokers.

what the agent holds

the user's token — unused for storage — **and NO S3 credential at all.**
The credential lives in the worker, in a system namespace, on a loopback door the agent container cannot reach. A compromised agent gets the tree it already had — not the bucket.

the door — flint-s3-broker

validates the POD's ServiceAccount token: kubelet-minted, pod-bound, audience s3.csi.chert.us
1 · TokenReview, ONLINE — a deleted pod's token dies within 60 s, not at exp
2 · a session name matching a LIVE registration the plugin made for this pod-uid + CR — a pod cannot self-mint it
3 · the CR lists the SA in consumers, in the TOKEN's namespace

what S3 sees

short-lived keys scoped to the project's prefix, minted per pod.
Never the user. Never a standing key held by anything a tenant can reach.
The broker's rest backend is the ONE seam where a user-scoped token could be honoured — it carries the pod's identity today.

what isolates user A from user B

The prefix, and the lease.
one workspace = one prefix and the keys are scoped to it; the subtree lease admits exactly ONE writer and lives in the BUCKET, so it holds across clusters; consumers lists who may mount it — absent means DENY

flint-passthrough

the same identity chain as flint-lean

Same broker, same TokenReview, same consumers gate.
The two front ends differ in what the worker RUNS, not in who is trusted. What differs is what happens INSIDE the pod.

what the agent holds

the user's token — unused for storage — **and NO S3 credential at all.**
The pod is given nothing: no sidecar, no label, no privilege, no webhook — just a csi: volume naming a CR. The smallest blast radius of the four, and the least it can promise.

the door — flint-s3-broker

identical to the lean lane: online TokenReview, a registration binding the pod cannot self-mint, and the CR's consumer list.
Every issuance leaves an audit line:
`(ns, sa, pod-uid, cr, expiry)`
One broker per cluster, again — every cluster's broker points at the same store, so the bucket still trusts one principal.

what S3 sees

short-lived keys scoped to the bucket and prefix in the CR.
And inside the pod: ONE uid.
The mount reports a single owner for every file, because NodePublish never sees the pod's securityContext — two processes, or two users, are indistinguishable at the mount.

what isolates user A from user B

The prefix and the CR — nothing finer than a POD.
consumers decides which ServiceAccounts may mount it; the keys are scoped to the prefix; there is no per-user view, so give each user their own CR and their own pod

flint-forge

one door, one server pod per repository

The pod's identity, proven to the CLUSTER rather than to the store.
TokenReview at the door — so a pod in another cluster carries nothing this door can verify. Cross-cluster agents fall back to the human bearer path.

what the agent holds

the user's token — unused — **and a projected ServiceAccount token, audience forge.chert.us,**
presented as the Basic password by a helper that re-reads it on every call. No bucket credential of any kind: those land on the syncer container of the server pod, only.

the door — flint-hub-gateway, Door::Git

TokenReview per session, cached ≤ 60 s by token hash; refusals cached, transport failures not.
spec.consumers must list the ServiceAccount; a credential-less peer is answered 401 before any wake.
`X-Remote-User: <sa> → REMOTE_USER`
read by pre-receive and by the syncer: the same principal, two enforcers, one policy document.

what the server sees

a ServiceAccount — which many pods share.
agentPattern bounds a branch NAME, not its owner:
one pod can push over a sibling's agent/* branch.
Protected refs, refs/for proposals and non-fast-forward refusal bind per principal; the server never sees the bucket credential the agent never had.

what isolates user A from user B

The repository: its pod, its prefix and claim, its consumers, its branch policy.
The X-Remote-User boundary is a NetworkPolicy admitting only the door — opt-in (door.namespace), and only where the CNI enforces it.
Otherwise reaching the port is the authorisation.

flint-lite · flint-lean · flint-passthrough · flint-forge · hazard / trust boundary

The answer, for all four, is that the user's token does not reach the data path. Every front end authenticates the workload, never the human behind it — so the fact that each agent may carry a different token is invisible to flint. For lean, passthrough and forge that is a feature: the enforced identity is the pod's ServiceAccount, kubelet-minted and pod-bound, so token churn upstream changes nothing. For lite it is a gap: **nothing on the NFS wire carries a user identity, and there is no field to add one to.**

flint-lite authenticates nobody on port 2049. AUTH_SYS means the client asserts its own uid and the server takes it, and the POSIX check is not a backstop either: `security.enforcePermissions` defaults to `false`, which evaluates the mode, logs the answer, and allows the operation anyway — and even enforced, the hub holds `CAP_DAC_OVERRIDE`. What `sec=sys` buys is identity, so `ls -l` tells the truth; it is not a boundary between tenants sharing a hub. So isolation must be drawn outside the hub, which the one-hub-per-project model already does: **one CR, one prefix, one coherence domain, and a real network boundary around it.**

flint-lean and flint-passthrough share one identity chain, and it is a strong one. The broker verifies the pod's token with an *online* TokenReview — a deleted pod's token dies within 60 seconds rather than at `exp` — requires a session name matching a live registration the node plugin made for that pod-uid and CR, which a pod cannot self-mint, and checks the CR's `consumers` list in the token's own namespace, where absent means deny. One broker per cluster is required, because TokenReview runs against the local API server: clusters meet in the bucket, not at each other's brokers. Both also accept an `ambient` mode with no broker at all, in which the worker uses the node's own cloud identity chain; the enforced identity is then whatever the platform enforces, and flint judges nothing. A web-identity exchange is lean-only: Mountpoint has no web-identity provider, so passthrough refuses that mode rather than falling back to the node's role and looking as if it had worked. **flint-forge borrows the shape at its door** — the same TokenReview, cached at most 60 s by token hash, the same `consumers` gate — and then carries the ServiceAccount to the server as `X-Remote-User`, which `pre-receive` and the syncer both read as the principal. The honest limits: the principal is the ServiceAccount, which many pods share, so a branch pattern bounds a branch's name and not its owner; and anything that can reach the server's port can set that header itself, which is why the boundary is a NetworkPolicy admitting only the door — rendered only when `door.namespace` is set, and enforced only where the CNI enforces it.

Fleet shape and blast radius

Two practical questions for anyone running agents on more than one cluster: what do I have to install in each of them, and if one agent pod is fully compromised, what does the attacker now have? The four front ends answer both differently, and the cheapest to install is not the one with the smallest blast radius.

flint-lite — one hub, N clusters

the HUB cluster — install here, once per project

flint-lite-operator (or the chart, one release per hub); the hub Deployment, Service, ConfigMap and PVC it renders; optionally flint-hub-gateway — one door holding ONE credential for the fleet
This cluster is now a dependency of every other one.

every CONSUMER cluster — install NOTHING

The data path is the node's own kernel NFS client: no driver, no DaemonSet, no CRD, no flint image. At most a PV and PVC, or the stock nfs-subdir provisioner, which never carries a byte.
The cheapest fleet onboarding of the four, by far.

what it costs you

One pod is the coherence authority, so one pod is also the single point of failure and the NIC ceiling. A hard NFS mount against a suspended hub HANGS — something must write the wake annotation before an address is handed out.

flint-lean — N syncers, one bucket

EVERY cluster — install the same three things

flint-s3-csi — the node DaemonSet, the workers namespace and its admission policy; flint-s3-broker — one per cluster, because TokenReview runs against the LOCAL API server; flint-lean — the CRD plus a thin controller

and no cluster depends on any other

The clusters never talk. They meet in the bucket, and the arbitration that keeps them honest lives there too — a CAS on the subtree lease no cluster can route around.
Single-writer holds ACROSS clusters, which is the one guarantee lite's admission check cannot make.

what it costs you

A per-cluster install kept in version step, and a broker per cluster to configure and watch. A lean volume still mounts with its controller absent — the syncer claims the lease itself — so the controller is an operability component, not a data-path one.

flint-passthrough — N mounts, one bucket

EVERY cluster — install two things

flint-s3-csi — the same node DaemonSet and broker as lean; one install serves both front ends. flint-passthrough — the CRD, and nothing else: no workload in the chart, no controller behind the CR.
Nothing about a passthrough mount converges, so there is nothing to reconcile.

the fleet story is: there isn't one

Pods in twenty clusters can mount one prefix and none of them coordinates, because none has anything to coordinate. S3 is already a multi-cluster service.
Scaling out is free precisely because nothing is shared except the objects.

what it costs you

Everything the other three provide. No POSIX, no rename, no boundary, no manifest, no single-writer rule. Two pods writing one key race, and the loser's write is simply gone — with no cell recording that it happened.

flint-forge — one door, one pod per repository

the HOME cluster — install here

flint-forge-operator and the FlintRepo CRD; per repository, the Deployment, headless Service, ConfigMap and — when door.namespace is set — a NetworkPolicy it renders; the door: door.deploy=true, or a lite gateway that has the git arm
Agents in this cluster need two lines of pod spec and stock git.

another cluster — an identity problem, not an install

Nothing of flint's is in the data path, but the door proves a token against ITS apiserver, so an agent elsewhere carries nothing it can verify. The bucket is a rendezvous for READERS only: a dumb clone works with the server down; a second server is fenced into a straggler that exits.
Cross-cluster agents fall back to the human bearer path. Not drilled.

what it costs you

One pod per repository in the data path — egress binds before CPU on a clone storm, and the lever (bundles from S3) needs the CLIENT to opt in. The server runs as root in both images.

S3 — the one thing every cluster, every front end and every fleet shape has in common. It is where the data lives; everything above is a way of reaching it.

Blast radius — an attacker owns one agent pod. What do they now have?

Assume the agent container is fully compromised and holds the user's token. This is the question the token cannot answer, because storage never sees it.

flint-lite — the whole share

The pod already has the mount. AUTH_SYS means it asserts its own uid, and the default POSIX check LOGS rather than refuses — so it reads and writes every file in the share, as anyone. It cannot reach the BUCKET: the hub holds that credential. It can reach any other hub its network can route to, and the next hub cannot tell it apart from a legitimate client.
Containment = one hub per project + a real network boundary. Everything sharing a hub shares a fate.

flint-lean — the workspace, not the bucket

The pod has the tree it was already working in and no S3 credential — the worker does, in another namespace, behind a loopback door the agent cannot reach. The attacker gets the files, published under the workspace's own prefix; not the bucket, other workspaces, or a credential that outlives the pod.
The honest ceiling: a deposed syncer's writes land as UNCITED versions a raw-key reader can still see — refused at the control plane, not fenced on the data plane.

flint-passthrough — the prefix, as mounted

The pod has whatever the mount has: the prefix in the CR, at the CR's readOnly setting, with keys that are short-lived, scoped, and never in the pod's env. The tightest of the four, for a dull reason: there is nothing else there to take.
Scope the CR and you have scoped the breach; readOnly: true is a real control. The residual is the one uid: pod-level isolation is user-level isolation.

flint-forge — a ServiceAccount's pushes

No bucket credential of any kind — those sit on the syncer container of the server pod — and no privilege: no sidecar, no /dev/fuse, no webhook. What it gains is the pod's projected token: push as that ServiceAccount to repositories listing it, bounded by policy to agentPattern branches and refs/for proposals. Revocation is pod deletion or SA rotation, felt within the ≤ 60 s review cache.
Not smaller: a shared ServiceAccount reaches every sibling's agent/* branch, and the server pod runs as root.

flint-lite · flint-lean · flint-passthrough · flint-forge · data path · hazard / trust boundary

flint-lite has the cheapest fleet onboarding and the widest blast radius. Consumer clusters install nothing at all — the data path is the node's own kernel NFS client — but the hub cluster becomes a dependency of every other one, and a compromised pod already holds the mount. With AUTH_SYS and a log-only permission check, it reads and writes every file in the share as anyone. It cannot reach the bucket, because the hub holds that credential and the pod has none; that much is genuinely contained. Everything sharing a hub shares a fate, so size hubs by the blast radius you are willing to accept, not only by capacity.

flint-lean and flint-passthrough cost a per-cluster install and buy a much smaller radius. Each cluster runs the same CSI node driver and its own broker; no cluster depends on any other, and they meet only in the bucket — where the lean lease is a CAS no cluster can route around, which is the one guarantee lite's admission check cannot make. A compromised lean pod gets the tree it was already working in, published under its own prefix; it does not get the bucket, other workspaces, or a credential that outlives the pod. **The honest ceiling is that lean refuses a deposed writer at the control plane, not on the data plane:** its writes land as uncited versions that no coherent reader resolves, but a raw-key reader can still see them. Passthrough is the tightest, for a dull reason: there is nothing else there to take. The pod gets the prefix in the CR at the CR's `readOnly` setting, with short-lived scoped keys it never sees.

flint-forge installs in one cluster and makes the other clusters an identity problem rather than an install. The home cluster runs the operator, the door and one small pod per repository; agents there need stock git and two lines of pod spec. Nothing of flint's is in the data path elsewhere — but the door proves a token against *its* apiserver, so an agent in another cluster carries nothing it can verify, and falls back to the human bearer path. The bucket is a rendezvous for readers only: a dumb-protocol clone works with the server down, and a second server is fenced into a straggler that exits. A compromised forge agent gains no bucket credential and no privilege; it gains its ServiceAccount's push rights, within policy, revoked by pod deletion within the review cache. Not smaller: a shared ServiceAccount reaches every sibling's `agent/*` branch, and the server pod runs as root.

S3 — the durable store

Underneath all four front ends there is one durable store, and it is S3. Everything above it — the hub's disk, the lean workspace, the FUSE mount, the forge clone and its server's local repository — is a way of reaching bytes whose home is the bucket. Two views of those bytes exist, with different contracts, and confusing them is the most common way to get hurt here.

The LIVE view — strong, and never in the bucket

a hub's port 2049 and its file API · a lean workspace's local disk · a forge clone's working tree, and the server's local repository
close-to-open · byte-range locks · atomic rename · no RPO lag
Reads and writes land on the working set, through whatever holds coherence for it — the hub process, the agent's own kernel, or git.
Who reads it: front doors, editors, agents — anything touching a tree that is being changed right now. The hub's live view offers one thing no real S3 can: assembly under a temp name completed by an ATOMIC RENAME under a deny-both OPEN. A lean or forge live tree is visible to exactly one pod.

The PUBLISHED view — the bucket, and the durable copy

RPO-consistent snapshots per subtree, made of untorn whole-file generations — or, for forge, a bare repository whose refs move as ONE CAS'd object.
whole objects · versioned or content-named · read-only against a live subtree
A reader never sees half a file or half a batch. It may well see a file from before your last write — that is the contract, not a defect. Anything that is not a mount reads this view as plain S3, with no flint server in the read path at all.
Do not write into a live subtree from the console: a hand-made PUT under a prefix a hub, syncer or server owns is overwritten at the next flush, reported as a conflict, or silently uncited.

Inside the prefix — the bytes, and the cells that make concurrency safe

Every cell below exists to answer one question that cannot be answered from the object listing alone.

flint-lite writes

`<prefix>/<path>`
the published generations
`.flint/epoch`
who is the hub right now? — a lease
`.flint/manifest`
what is the whole tree? — DR from one GET
`.flint/owner`

flint-lean writes

`<prefix>/files/<path>`
the workspace, file for file
`.flint/lean/epoch · claim`
who may write this subtree? — one holder, by CAS, across every cluster
`.flint/lean/current`
what does the last boundary contain?
`.flint/lean/inbox · conflicts/`

flint-passthrough writes

`<prefix>/<key>`
`the objects, and nothing else`
There is no control namespace here, and that is the entire feature.
A prefix owned by somebody else's tooling stays exactly as it was; flint claims no lease and leaves no trace. You also get no answer to any question on the left.

flint-forge writes

`git/objects/pack/*`
immutable, content-named — never overwritten
`git/snapshot`
the refs, packs, bundles and exported commit — ONE CAS'd pointer
`git/epoch · git/claim`
who is the server? whose prefix?
`lfs/objects/<oid>`
and, under its OWN prefix, the export

Reserved, and enforced

.flint is control state: a client file under it can never shadow a control object, and a NESTED .flint/ is another share's namespace — never tiered into this one. Both are tested, because both were once a way to overwrite a live share's lease.
The file-shaped formats are mutually convertible, and forge's export is one of them: choosing today does not strand data.

What “durable” actually means here — the recovery point, per front end, in the language of a hard pod kill

flint-lite — RPO is the flush cadence

The PVC is a working set the hub flushes on a timer; losing the disk is a REBUILD from the bucket, not a loss. A cold read hydrates on demand and both doors say so: NFS4ERR_DELAY, or 503.
Hibernation deletes the PVC, so it is VERIFIED, never assumed:
the operator scales back to one, polls until the hub reports rpoClean, and only then deletes. A share with no bucket reports rpoClean: null — a refusal, not a pass.

flint-lean — RPO is the last barrier

A hard kill loses at most floorSecs (default 60 s); a graceful stop loses nothing, because preStop runs a final barrier. A restart over a live tree RESUMES from its baseline.
Publish at a breakpoint and the RPO becomes zero, knowably:
`echo > .flint/publish → publish.ack = ok`
means the bytes are in S3 — the difference between “probably durable” and “durable, and I was told so”.

flint-passthrough — there is no RPO

Nothing is buffered on flint's behalf, so nothing can be lost by flint. A completed PUT is durable the instant S3 says so; an interrupted one did not happen.
The strongest durability story of the four — and strongest because flint is not in it.
The trade: a write is a whole new object, there is no rename to make it atomic against a reader, and two writers racing one key resolve as S3 resolves them — last writer wins, unrecorded.

flint-forge — RPO is the last acknowledged push

No path acknowledges a push the bucket does not hold: the pack is uploaded, the snapshot CAS lands, the ref transaction applies, and only THEN is ok reported. A crash between the CAS and the report fails the push at the client; the restart restores from the snapshot and passes fsck.
The unit is a push. Uncommitted work has no RPO —
git's contract, which forge keeps; a harness that wants one runs a snapshot script in its own pod, force-pushing refs/wip/<pod>.

flint-lite · flint-lean · flint-passthrough · flint-forge · hazard / trust boundary

The live view is strong and is never in the bucket. It is a hub's port 2049 and file API, a lean workspace's local disk, or a forge clone's working tree: close-to-open, byte-range locks, atomic rename, no RPO lag. The published view is the bucket: RPO-consistent snapshots per subtree, made of untorn whole-file generations — or, for forge, a bare repository whose refs move as one CAS'd object, so a reader never sees half a batch either. A reader of the bucket never sees half a file, and may well see a file from before your last write — that is the contract, not a defect. **Do not write into a live subtree from the console:** a hand-made PUT under a prefix a hub, syncer or server owns is overwritten at the next flush, reported as a conflict, or silently uncited — and under an export prefix, as page 12 shows, it simply stands.

Every control cell exists to answer a question the object listing cannot. Who is the hub right now, and may I become it — the epoch lease. What does the last coherent boundary contain — the manifest, which also makes disaster recovery a single GET rather than a crawl. Who owns this prefix, and did state loss change that — the claim and owner cells. Forge's `git/snapshot` answers all three of its own at once — the refs, the packs, the bundles, the exported commit — and its `git/epoch` is the same kind of lease under a different key, which is exactly why two products on one prefix never contend. Passthrough writes none of them, which is why it can be pointed at a prefix another system already owns and leaves no trace when the pod goes away; it also gets no answer to any of those questions.

The recovery point differs, and it is worth stating in the language of a hard pod kill. Lite loses up to the flush cadence, and its PVC is a cache — losing the disk is a rebuild. Lean loses at most `floorSecs` (default 60 s), zero on a graceful stop, and zero-and-acknowledged if the agent declares a boundary. Passthrough has no recovery point at all, because nothing is buffered on flint's behalf: a completed PUT is durable and an interrupted one did not happen. Forge's is the last acknowledged push — no path acknowledges a push the bucket does not hold, which eight mid-push pod kills on EC2 left standing with `fsck --strict` clean — and uncommitted work has none, which is git's contract. The converse exists and is not a loss: a client cut after its pack is in can be told *failed* while the bucket holds the push, and its retry finds the ref already there; the formal model carries that transition as a required one.

Security posture

Seven questions, asked of each front end in turn: how a writer proves who it is, what protects the wire, what a merely-reachable peer can do, what keeps tenants apart, whether two sessions on one project can hold different rights, what an attacker who owns an agent pod gains, and whether the posture fails closed under operator error. Every cell is a claim the pages before this one already made, or a verified cell of the approach radar; none is a score.

dimension	flint-lite	flint-lean	flint-passthrough	flint-forge
Who a writer must prove it is before storage accepts an operation	**Nobody, on port 2049.** AUTH_SYS: the client asserts its own uid and the server takes it. Identity is self-declared, not proven; the file API's bearer is per-share. krb5 exists in the server and is not surfaced by the chart.	**The pod's ServiceAccount, online.** kubelet-minted, pod-bound, audience s3.csi.chert.us; TokenReview at the broker, a registration the pod cannot self-mint, and the CR's consumers list. The agent container holds no key at all.	**The pod's ServiceAccount, online.** The identical chain: same broker, same TokenReview, same registration binding, same consumers gate, same audit line. Inside the pod every process is one uid.	**The pod's ServiceAccount, at the door.** A projected token, audience forge.chert.us, as the Basic password; TokenReview per session, cached ≤ 60 s by hash. The principal is the SA, which many pods share — agentPattern bounds a branch name, not its owner.
What protects the wire confidentiality and integrity in transit	**Cleartext RPC.** sec=sys carries no per-user authentication and no encryption; RPC-with-TLS is prospective. Reachability is the boundary, so draw it at the network layer — peering, security groups, a gateway.	**Nothing of flint's crosses the network.** The agent writes local disk; the worker publishes to the S3 endpoint over its TLS. The credential door is a loopback socket on the node; the broker is reached in-cluster and answers keys, not bytes.	**S3 over TLS, unmediated.** mount-s3 talks to the endpoint directly; the FUSE descriptor crosses a node-local socket; keys arrive on a loopback door, never as pod env. No flint server is on the read path.	**Two cleartext hops, as built.** HTTPS at the door is a line in the diagram; nothing in-tree terminates it, and the git container's runner listens on 8080 with no TLS. The pod's audience-bound token rides both hops as a Basic password — a network observer inside the cluster holds a live credential. Server-to-S3 is TLS.
What a reachable peer can do with no credential at all	**Open a full NFSv4.1 session.** Port 2049 needs no credential: the defence is caps — state-table quota, slot cap, idle deadline, READ clamp — not authentication. A merely-reachable peer holds a stateful session.	**Very little.** No flint door sits on the data path; the broker refuses without a valid TokenReview, and S3 refuses unsigned requests. What it can flood is the broker's TokenReview against the local apiserver.	**Very little.** The same: the reachable service is the broker, and the store refuses unsigned requests. The node plugin's socket is kubelet's, not the network's.	**A 401 before any wake** — authentication precedes the repository's wake, so a peer cannot scale a parked repository up, and a refused token is cached. The lever it keeps: every novel bad token is one TokenReview; behind the door the runner sets no body limit and no timeout of its own (the door's idle bound is the one cut), and a wake holds a door slot up to 180 s.
What keeps tenants apart reading or writing another's data	**The network, and one hub per project.** No auth on 2049 means the boundary is reachability, which networkPolicy cannot draw for remote clusters; one CR, one prefix, one coherence domain, and everything sharing a hub shares a fate.	**The prefix, its claim and its lease.** Keys are scoped to the workspace prefix; the claim refuses a foreign project on a reused prefix; the lease admits one writer and lives in the bucket, so it holds across clusters; consumers absent means deny.	**The prefix and the CR.** consumers decides which ServiceAccounts may mount it, keys are scoped to the bucket and prefix in the CR, and the CR lives in the pod's own namespace. Nothing finer than a pod: one uid per mount.	**The repository.** Its own pod, its prefix with a claim that refuses a foreign projectId, arbitration of overlapping prefixes at the operator — export prefixes included — consumers at the door, and a per-principal branch policy read by two enforcers. **X-Remote-User is the boundary, and it is a NetworkPolicy: opt-in, and only where the CNI enforces it.**
Can two sessions differ? in rights: one project, one reader, one writer	**No — only a client-side ro mount.** A read-only claim gets ro, and the CSI publish forces it over operator options; but the server authenticates nobody, so any pod that reaches 2049 mounts rw and asserts uid 0. enforcePermissions checks a claimed uid: defence against accident, not against a session that wants to write.	**No.** A workspace is one writer: the publish path hardcodes read-write, the CRD has no readOnly, and the credential is one key per project. A second lean pod on the same prefix does not become a reader — it waits to become the writer. The read-only session is a passthrough mount of files/, readOnly (page 12).	**Yes, per pod.** Two pods on one CR, one declared readOnly: --read-only on mount-s3 plus a read-only bind mount, and the pod never holds the key. The broker gates by consumers and hands both pods the SAME scope — one static key or one role ARN. The REST backend is told the ServiceAccount, so an external service can scope the key per principal; nothing in-tree does.	**Yes, per ServiceAccount.** consumers grants read; the branch policy decides writes per principal, in pre-receive and again in the syncer. A reader is listed in consumers with every ref protected and only the writer's SA in pushers; refs/for stays closed without mergeInto. **Until a branches block exists, every consumer may push.**
What an owned agent pod gains and how fast the gain is revoked	**The whole share, as anyone.** It has the mount, asserts any uid, and the permission check logs by default. Not the bucket — the hub holds that credential. Nothing per-client to revoke; containment is the network boundary.	**Its own workspace, and nothing that outlives it.** No S3 credential in the agent container in either deployment mode; the sidecar holds it, unprivileged. Revocation is killing the token or the pod. Ceiling: a deposed writer's uncited versions remain readable by a raw key.	**The prefix, as mounted.** Short-lived scoped keys it never sees, at the CR's readOnly setting; no sidecar, no privilege. Scope the CR and you have scoped the breach; the residual is the single uid.	**Its ServiceAccount's push rights.** No bucket credential — they land on the syncer container only — and no privilege. It can push as that SA to repositories listing it, within policy; revoked by pod deletion or SA rotation within the review cache. Ceiling: a shared SA reaches every sibling's agent/* branch, and the server runs as root.
Does it fail closed? secure defaults, refusal over silent degradation	**No, by default.** security.enforcePermissions is false — evaluate, log, allow — and even enforced the hub holds CAP_DAC_OVERRIDE. SECINFO now advertises AUTH_SYS first and AUTH_NONE last, pinned by a test, so a stock mount negotiates sec=sys.	**Detection becomes refusal.** A syncer that lost its lease answers refused-fenced rather than publishing; the plugin refuses to clean up a published tree; the chart refuses to render on credential misconfig; gated mode is refused without a lag bound.	**Deny is the default.** consumers absent means deny; the ValidatingAdmissionPolicy — not the namespace label — fences the workers namespace to the plugin's own node; a dead mounter strands the pod on ENOTCONN rather than serving stale bytes.	**Closed where it counts, open on two defaults.** An unparseable policy refuses; a MISSING pre-receive does not open the repository; a snapshot naming a missing pack refuses to serve. **No door.namespace renders no NetworkPolicy, and an absent branches block is permissive: both documented, neither refuses.**

Read it down a column. The identity chain lean and passthrough share is the strongest thing on the page, and forge borrows its shape at the door; the hub's is the weakest, and its boundary is drawn with the network. Every cell is the deck's own claim, made on the pages before this one, or the approach radar's verified Security cell.

flint-lite · flint-lean · flint-passthrough · flint-forge · hazard / trust boundary

Read it down a column. The chain lean and passthrough share is the strongest thing on the page: the pod's ServiceAccount, kubelet-minted and pod-bound, verified *online* by a broker that also demands a registration the pod cannot self-mint and a `consumers` entry in the token's own namespace, exchanged for short-lived keys scoped to one prefix that the agent container never holds. Forge borrows the first half of that shape at its door — the same TokenReview, cached at most 60 s — and then differs in what happens next: the principal is carried to the server as a header, which makes the network between door and server a trust boundary, and nothing in-tree terminates TLS, so the token itself rides both hops in the clear. The hub's is the weakest: port 2049 proves nothing, the permission check logs by default, and the boundary has to be drawn with the network, which `networkPolicy` cannot do for remote clusters.

None of the four knows the human. A user's read-only role becomes a read-only session only when whatever launches the pod picks the ServiceAccount, or sets the mount flag, from that role. Kubernetes lets anyone who can create pods in a namespace use any ServiceAccount in it, so the reader and the writer need separate namespaces or an admission policy that binds user to ServiceAccount; flint never sees that decision. Given it, the row above holds: two sessions with different rights on one project work on forge and on passthrough, are a mount flag the server cannot back on lite, and on lean are a composition with a passthrough reader. Three of the four were put on the wire on 2026-09-05: on forge a reader and a writer on one repository, refused and allowed per ServiceAccount, with the `X-Remote-User` boundary shown to hold only behind the operator's NetworkPolicy — delete it and a forged header merges; on passthrough two pods on one mount, separated only by the read-only flag, both drawing the same key; and on lite a krb5p mount that proves the user and enforces none of their rights, even in enforce mode. Lean's single writer is by construction, undrilled.

Two defaults are worth knowing before the first install. On lite, `security.enforcePermissions` is `false`, and a stock mount negotiates `sec=sys`; that is identity, not enforcement. On forge, a chart with no `door.namespace` renders no NetworkPolicy, so the `X-Remote-User` boundary is silently absent, and a repository with no `branches` block is permissive, so any listed consumer may move `main`. Both are documented; neither refuses. Lean and passthrough fail closed where it counts — a syncer that lost its lease answers `refused-fenced` rather than publishing, an absent `consumers` list means deny, and the workers namespace is fenced by an admission policy rather than a label — and forge does too on its hooks: an unparseable policy refuses, a missing `pre-receive` does not open the repository, and a snapshot naming a pack the bucket lacks refuses to serve.

Composition — one bucket, several doors, one writer per prefix

The front ends are not alternatives to pick between once. A workload has several kinds of data, each kind has a best door, and one pod can hold all of them at once — a forge clone, a lean workspace, a passthrough mount, a hub share — because each lands in its own prefix of one bucket. This page states the rule that keeps that safe, and what the composition drills of 2026-09-04 found when the rule was tested rather than assumed.

ONE AGENT POD four attachments; a stock image with no flint code in it

/src — a git clone
code and history; a push is durable; a merge is a push to refs/for
the door → the repository pod

/workspace — a lean mount
the hot working set: plain local files, one writer, a boundary it can name
flint-sync → the bucket

/data — a passthrough mount
read-mostly datasets, weights, artifacts — the bucket as-is, readOnly
mount-s3 → the bucket

/shared — a hub share
what several pods must edit LIVE: coordination files, sqlite, cross-pod locks
the kernel NFS client → the hub

A → `repo/`
C → `ws/<agent>/`
D → `datasets/`
E → `shared/`

a build-farm pod, elsewhere
mounts B/files/ — forge's export of main — read-only through lean or passthrough and never runs git. That is the composition: several readers of one writer's prefix.
It writes nothing there. See the hazard on the right.

one bucket, five prefixes, one writer each

letters match the attachments on the left; the arrow is the only bridge

A repo/ — writer: the syncer
`git/objects/pack/*  git/snapshot`
`git/epoch · git/claim  lfs/objects/`

B repo-main/ — writer: the syncer, via the shipped flint-sync
`files/<path>  .flint/lean/{epoch,current,chunks/}`
a REAL lean workspace; readers: lean, and passthrough aimed at B/files/

C ws/<agent>/ — writer: the lean sidecar
`files/<path>  .flint/lean/{epoch,claim,current}`

D datasets/ — writer: whoever owns them
`<key>  — objects and nothing else; passthrough reads`

E shared/ — writer: the hub
`<path>  .flint/epoch · manifest · owner`

A → B is the export: after a push moves main, the syncer materialises the tree and runs flint-sync barrier over it. Nothing else crosses a prefix boundary, in either direction.

One writer per prefix — where it is a mechanism, and where it is not

Within a product, a mechanism:
an epoch lease plus one CAS'd pointer — forge's on git/epoch, lean's on .flint/lean/epoch. A second writer of the same product does not acquire: it observes the standing lease and waits, superseding only after six quiet polls have shown the holder dead; drilled both ways.
Across products, a convention: the cells are disjoint.
Point forge and lean at ONE prefix and they never meet — both acquire epoch 1 under different holders; no 412, no fence, no log line (drill C1). The operator's arbitration reasons over FlintRepos only: it refuses an export aimed at another repository and sees nothing involving a lean CR.
The one arbitrated direction: a read-write lean sidecar on the EXPORT prefix contends on the same cell — and until today that wedged the repository, because the syncer awaited the blocked export inline with its own heartbeat (C2; now bounded by a timeout with backoff — pushes still stall for that long).

An export is a mirror that is never repaired

The barrier computes uploads and deletes from a LOCAL scan against a LOCAL baseline; the only remote thing it reads is the pointer's etag. A foreign write into B moves no pointer and changes no local file, so no later export notices it (C3).
A reader that cannot verify takes the foreign bytes (C4):
a passthrough or lite mount has no manifest to check against and serves them. A reader that verifies now refuses: the export marks its manifests as published by a sole writer, and lean's checkout refuses an object off its citation — after the drill first found it adopting. Detectable, not self-healing.
A foreign DELETE is refused loudly by lean and never restored (C5).
For the unverifying reader an overwrite stands silently while a delete is refused loudly: the milder-looking operation is the dangerous one.
So a read-write mount over an export prefix is unsupported, and readers of an export are readers.

Pick one per data class, not one front end per team

The choosing page scores each column as if it stood alone. In practice a workload has several kinds of data, each with a best door — and each in its own prefix.

code, with history
durable on push, reviewed by policy, merged by the server; many agents proposing to one main

the hot working set
a repo-sized tree at disk speed, for one writer, with a boundary it can name; the cheapest thing that works

datasets, weights, artifacts
read-mostly, large, or owned by another system; the bucket as-is, and nothing to keep in step

checkpoints and large binaries
LFS, client-to-store — or a passthrough prefix, if nothing needs them in history

coordination, sqlite, live RWX
the only place several pods edit one tree with locks that mean something; size the hub by its blast radius

a legible copy of main
the export: read it through lean or passthrough with no git in the pod — and never write to it

flint-lite · flint-lean · flint-passthrough · flint-forge · data path · hazard / trust boundary

The composition is in the reading, never the writing. The legal shape has one writer per prefix: the syncer writes the repository's prefix and, through the shipped `flint-sync`, its export prefix — a real lean workspace, `files/` beside `.flint/lean/` — which lean and passthrough readers mount read-only with no git in the pod; a lean sidecar writes its own workspace prefix; the hub writes its share; a passthrough mount reads a prefix somebody else owns — aimed at the export's `files/`, so it never sees the control directory beside it. The only bridge between prefixes is the export, and it runs one way. Every file-shaped front end publishes plain whole-file objects, so a passthrough reader can read what any of them published; that is the substrate the whole page rests on, and page 10 already drew it.

One writer per prefix is a mechanism within a product and a convention across them. Within forge, or within lean, a second writer of the same prefix does not acquire: it observes the standing lease and waits, superseding only after six quiet polls have shown the holder dead — drilled both ways. Across the two, the cells are disjoint: forge arbitrates on `git/epoch`, lean on `.flint/lean/epoch`, so a forge server and a lean sidecar pointed at one prefix both acquire epoch 1 under different holders, and nothing 412s, fences, or logs a line (drill C1). The operator's arbitration reasons over `FlintRepo`s only, on exact prefix equality: it refuses an export aimed at another repository's prefix and sees nothing involving a lean CR. The one direction that *is* arbitrated — a read-write lean sidecar on the export prefix contends on the same cell — used to wedge the repository itself, because the syncer awaited the blocked export inline with its own heartbeat; that is fixed by a bounded timeout with backoff (C2), and the heartbeat has since moved to its own task, so a blocked export can no longer cost the lease — the residual is that pushes still stall for that long.

An export is a mirror that is never repaired. The barrier computes what to upload and delete from a local scan against a local baseline, so a foreign write into the export prefix is not noticed by any later export (C3). Every reader that cannot verify still takes the foreign bytes — a passthrough or lite mount has no manifest to check against. A reader that verifies now refuses, because the export marks its manifests as published by a sole writer and lean's checkout refuses an object off its citation; the drill first found lean adopting it, and `78c4445e` is the fix (C4). The divergence is detectable but not self-healing — and only for a manifest published with that mark: an export written before the fix stays adoptable until it publishes again. A foreign *delete* is refused loudly by lean and never restored (C5). For the reader that cannot verify, an overwrite stands silently while a delete fails loudly: the milder-looking operation is the dangerous one. So a read-write mount over an export prefix is unsupported, and readers of an export are readers. The decision of record, 2026-09-05, is to leave the export unrepaired: the suite records that and three neighbouring conditions as accepted, goes green at 45 passed, 0 failed, 8 accepted, and fails if an accepted condition ever stops reproducing. What did ship is detection: at claim time each product probes the other's epoch cell — `git/epoch`, `.flint/lean/epoch` — and names the collision, because prevention belongs to whatever assigns prefixes, and an in-band refusal would be an outage the first time a stale cell outlived its workspace.

Choosing

Eleven dimensions, four columns. Read it down a column rather than across a row: each front end is coherent with itself, and the rows that look like weaknesses are usually the price of a strength two rows up. Then read it against page 12, because the last row names a data class, not a team.

dimension	flint-lite	flint-lean	flint-passthrough	flint-forge
Technology in one word: what carries the bytes	**NFS.** a hub pod serves the tree over NFSv4.2; the node's kernel client carries every byte	**Sync.** the agent writes a local directory; a sidecar copies changed bytes to S3 with the plain S3 API, after the fact. No FUSE anywhere: nothing intercepts a file operation, nothing mounts the bucket	**FUSE.** Mountpoint for S3 intercepts every file operation in the pod and translates it into object requests	**Git.** real git, served per repository, with S3 as its only durable state
What the pod sees the shape of the thing	a shared NFS mount served by one hub pod that every client reaches over the network — one tree, many pods, live	plain files on local disk, checked out before the first line runs and published back on a boundary — one pod's own tree	an S3 prefix presented as files by Mountpoint for S3 — objects, with a directory-shaped view over them	a git remote. The working tree is a local clone the pod owns; history, branches and a merge policy live at the server
POSIX fidelity what actually works	**Full, and shared.** byte-range locks, atomic rename, O_EXCL — across pods and clusters	**Full, and local.** a real filesystem, so git, sqlite and hard links work — for one pod	**None to speak of.** no rename, no append, no in-place write — at any setting, deliberately	**Full, and local — for tracked files.** the clone is a real disk; what is durable is what is committed
Concurrent writers per tree	**Many, coordinated.** the only front end where two pods can edit one directory and both be right	**Exactly one, enforced.** a second syncer is REFUSED, not merged — several agents integrate through git instead	**Many, uncoordinated.** two writers on one key race; the loser's write is gone and nothing records it	**Many, serialised by the server.** one syncer per repository; a stale push is refused by name; a merge is a push to refs/for/<target>
What a reader sees the consistency contract	the live tree, strongly — close-to-open, no RPO lag, through the coherence authority	the last PUBLISHED boundary, never your last write; gated mode makes it all-or-nothing	whatever S3 currently holds, with S3's own consistency and nothing added	on fetch, every acknowledged push — served by the write authority, never stale, never from S3; between fetches, a snapshot with no signal
Across clusters the fleet shape	ONE hub, mounted from N clusters. Works unchanged — but needs unique client names, a network boundary and a keepalive	N independent syncers, one bucket. Clusters never talk; the lease lives in the bucket, so single-writer holds ACROSS clusters	N independent mounts, one bucket. There is no fleet problem, because nothing is shared to get wrong	one server per repository; the door proves tokens to ITS apiserver, so remote agents need the human bearer path. The bucket is a rendezvous for readers only — undrilled
Identity that is enforced the user's token reaches NONE of these	**Nobody, on the data path.** AUTH_SYS: the client asserts its own uid, and the POSIX check defaults to log-only	**The POD's ServiceAccount.** kubelet-minted, pod-bound, verified ONLINE by TokenReview, checked against consumers	**The POD's ServiceAccount.** an identical chain to lean — same broker, same registration binding, same audit line	**The POD's ServiceAccount, at the door.** TokenReview per session, consumers per repository, then X-Remote-User to two enforcers of one policy
Isolating user A from B when both launch agents on several clusters	Give each project its own hub and prefix, and bound reachability outside Kubernetes. **Everything sharing a hub shares a fate.**	Give each user their own workspace prefix. Keys are scoped to it, the lease fences it, and consumers says who may mount it	Give each user their own CR and pod. The mount presents ONE uid, so pod-level isolation IS user-level isolation	Give each project its own repository and each agent its own branch pattern; a shared ServiceAccount reaches every sibling's agent/* branch
Installed per cluster in a consumer cluster	**Nothing.** the node's own kernel NFS client is the data path; at most a PV and PVC	the s3.csi.chert.us driver, a broker and the lean CRD + thin controller — in EVERY cluster; the broker must be local	the same driver and broker, and the CRD — in every cluster. One CSI install serves lean and passthrough together	**Nothing in the data path.** stock git plus two lines of pod spec; the door and the servers live in the home cluster
Recovery point what a hard kill costs	the flush cadence. The PVC is a cache — losing it is a rebuild from S3, not a loss	the last barrier: ≤ floorSecs (default 60 s), or zero and acknowledged if you declare one	**None — nothing is buffered.** a completed PUT is durable; an interrupted one did not happen	the last acknowledged push — ok means the pack and the snapshot are in S3. Uncommitted work: none, by git's contract
Choose it for a data class, not a team	**What several pods must edit LIVE.** shared workspaces, cross-pod locking, sqlite, a dataset too big for a pod's disk	**One agent's hot working set.** a repo-sized tree, full POSIX at local speed, durable at breakpoints	**The bucket, not a filesystem.** read-mostly datasets, weights, artifacts, or a prefix another system already owns	**Code, with history.** durable on push, a merge policy the server enforces, many agents proposing to one main; large binaries in LFS

Read the table down a column, not across a row: each front end is coherent with itself, and a row that looks like a weakness is usually the price of a strength two rows up.

Then read the composition page: one workload usually has several of these data classes, and each has a best door. Because the file-shaped formats are common and forge exports one, a volume can change its mind later.

The short form. Choose **flint-lite** for what several pods must edit live — shared workspaces, cross-pod locking, sqlite, a dataset larger than a pod's disk, or anything needing a filesystem two pods can both see. Choose **flint-lean** for one agent's hot working set: a repo-sized tree that fits on disk, full POSIX at local speed, and a durable point the agent can name. Choose **flint-passthrough** when you want the bucket rather than a filesystem — read-mostly datasets, weights, artifacts, or a prefix another system already owns that you want visible inside a pod without changing it. Choose **flint-forge** for code with history: durable on push, a merge policy the server enforces, many agents proposing to one `main`, and large binaries in LFS.

The multi-cluster and security rows are the ones most often decided late and regretted. If one user's agents must be isolated from another's, none of the four will do it for you inside the data path: the unit of isolation is the prefix, and the thing that enforces it is a network boundary around a hub, a broker scoping short-lived keys to a CR, or a door and a branch policy around a repository. Decide the prefix layout first and the front ends second — plural, because a workload usually has several data classes and each has a best door. **And because the file-shaped formats are common and forge exports one, a volume can change its mind later** — which is the strongest argument for starting with the cheapest bargain that works rather than the most capable one.